The Den — Tushar's Workspace

Complete functional and process description, handed over to Claude Design to create the whole Den UI. Generated 2026-10-06 from the codebase at /Users/work/TheDen.

> **How to read this.** Sections 1–3 describe what The Den is and the things it manages. Section 4 is the processes (the main part). Section 5 describes the current UI as it exists today. Section 6 is the visual system. Section 7 lists the full screen inventory, section 8 the open items and the design brief. Section 9 is the reference (API endpoints, settings, files). Anything marked **not built** does not exist yet.

Contents

1. What The Den is
2. Entities (the nouns the UI shows) and all statuses
3. Access, auth and global behavior
4. Processes (detailed workflows)
5. The current UI (navigation, screens, sheets, keys, gestures, notifications, responsive)
6. Visual system (tokens, style, motion, states)
7. Screen inventory checklist
8. Open items, gaps and design brief
9. Reference (endpoints, settings, files)

1. What The Den is

- **The Den** is a single-owner, always-on service that runs on the user's Mac (formerly called claude-queue). Its product name is **"Tushar's Workspace"**. The API runs at http://127.0.0.1:8787 and the dashboard at /dashboard.
- **Octopus** is the nickname for the worker and the whole AI system: one brain (the router that decides which model handles a request) with many arms (Claude Code in any registered project, plus API models such as OpenAI and Gemini). The mascot is an octopus drawn in code with six moods.
- **Core loop:** the user queues tasks per project. The Den runs them through Claude Code one at a time, moves to the next when one finishes, and resumes by itself after a usage limit.
- **Layers on top of the queue:** Octopus request routing and planning; a personal assistant (email, calendar, reminders); voice and Alexa; a lightweight CRM and project-management layer (clients, contacts, activities, follow-ups, timeline, phases and autopilot); email and push notifications; documents, notes and a pad.
- **It lets the user** trigger work from other devices (phone dashboard, SwiftBar, iOS Shortcuts, Alexa) instead of only from an editor.
- **Project plan:** CRM [done]; Email notifications [done]; **UI Setup [planned]** — this document is the input for UI Setup.
- **Platform:** phone first (bottom tab bar, swipe between pages, full-screen chat) with a sidebar layout from 900px up. The front end is plain ES modules with no build step. The older single-file dashboard still exists at `/dashboard/legacy`.

2. Entities and statuses

Entity	Fields and notes
Project	Name; folder path (can be none = "CRM only"); category (work/personal); status; pipeline stage; priority; health; start, due and completed dates; contract value and currency; billing type (fixed, hourly, retainer); owner (free text); tags (up to 30); links (up to 20, label + URL); color and logo background (#rrggbb); picture; client; hidden; active (paused); monthly API budget (default $20); pinned Claude account; autopilot mode and ceiling; default priority; notification toggles; allowed commands; deploy targets; auto-in-progress flag. A built-in project named **General** (workspace/general) receives requests with no project; it has no CRM record and cannot be hidden.
Task	Prompt, project, status, model, category, routing reason, priority, permission mode, kind (task or message), thread, queue position, step log, duration, tokens, cost estimate, unread flag, git branch/commits/files changed, tests seen, result.
Backlog to-do	Text, order, checked, important (star), priority, model, permission mode, images, phase. Not run until checked and "Run selected" is pressed.
Phase	Name (≤120), goal (≤2,000), description (≤4,000), position, status, start/end dates, blocked note, started/finished times.
Project note	Kind: guideline / requirement / note; text; images. Every task run for the project is given them. The first note is the "pinned note" on the Work tab.
Document	File up to 25 MB; sanitized name (≤120); mime; size; created time. Text files are given to models as context (up to 40,000 characters, 50 documents).
Client	Name (required, unique, ≤120), industry, website, address (≤500), notes (≤20,000), status (prospect default / active / past), color, logo. Computed: project count, active projects, contact count, value per currency (total of all non-lost projects; active = active + on_hold stages).
Contact	Optional client, name (required), role, email, phone (≥3 digits), notes, preferred method (email, phone, text, whatsapp, none). On a project a contact is the **key** contact (at most one) or a **stakeholder**.
Activity	Kind (call, meeting, email, note, decision, stage), occurred-at (not more than a year in the future), summary (≤4,000), outcome (≤2,000, optional), people involved (≤30), exactly one subject (project, client or contact). Stage activities are written by the Den, can be deleted but not edited, and carry from/to/by.
Follow-up	Title (≤200), notes, due date, optional due time (HH:MM), status (open/done), done-at, exactly one subject. Computed: due-at, days left, overdue, subject name.
Timeline item	Kind (deadline, milestone, task, presentation, demo, meeting, event), title (≤200), notes (≤4,000), date, optional time, custom alert days (≤60), status (open/done/cancelled), source (dashboard/octopus/api). Meetings add: contacts, guests (≤50 emails), duration (default 60, max 1,440), repeat rule, Google event id, invite link, Meet link, invite error.
Approval	Tool name, tool input (the command), status, risk read, steps so far, project's uncommitted diff, decision note.
Recipe	Name, prompt, target (any, all, or a project id).
Schedule	Prompt, label, project (null = every active project), cadence (once, daily, weekdays, weekly), time, weekday, date, model, priority, enabled.
Pad note	Title, body, optional project, optional meeting, saved file path, timestamps. Can be saved as a real Markdown file.
Reminder	Created by the assistant; pushed through ActivitySmith; listed and cancellable.

Claude account	Kind (mac login or token account), label, email, alias, color (8-color auto palette or chosen), org/plan, logged-in, active (default), usage windows (5-hour, weekly), limited-until, last hit, auth error, added-at, projects using it. The token is a secret and never shown.
API key	Provider (OpenAI/Gemini), label, last four characters, default flag, status/error. A built-in environment key is always listed first and cannot be removed there.
Model / Agent / Tool	Registries (models.json, agents.json, tools.json): provider, capabilities, ratings, context size, price, enabled, availability and limits.
Module	Optional feature in modules/<name>/ with a manifest; status running, disabled, blocked, failed, invalid.

All statuses and enumerations

Thing	Values
Task status	queued, running, rate_limited, paused, done, needs_review, error, cancelled
Task kind	task (runs to completion, changes files) / message (a question to a model; never changes files; "finished" = done and read; unread until read)
Lanes	agent (Claude Code in the project), api (plain OpenAI/Gemini call), assistant (email/calendar), control (instant commands)
Priority (tasks)	Urgent (1), Normal (0), Low (-1)
Backlog to-do status	pending, queued, running, rate_limited, done, error, needs_review, cancelled (queued/running/rate_limited = "in flight"; error and needs_review = "failed")
Project status	not_started, in_progress, need_info, on_hold, completed, archived
Pipeline stage (Board)	lead, proposal, active, on_hold, done, lost
Stage → status	lead, proposal → not_started; active → in_progress; on_hold → on_hold; done → completed; lost → archived
Status → stage	not_started → lead; in_progress, need_info → active; on_hold → on_hold; completed, archived → done. A project with no stage picked shows the stage derived from its status. Picking a stage moves the status; changing status moves the stage only if it no longer fits. Every stage change is logged as a "stage" activity ("Lead → Proposal").
Project priority	low, medium, high, critical
Project health	on_track, at_risk, off_track
Due state	none, done, overdue, due_soon (within 7 days), upcoming. "done" applies when completed or archived or a completed date exists.
Timeline health	none, on_track, at_risk, overdue, done (each with label, reason and next item)
Client status	prospect, active, past
Phase status	planned, active, paused, done (only one active per project)
Autopilot	manual, timeline, capacity
Approval status	pending, approved, denied, timed_out
Hero states (Home)	waiting, rate-limited, working, held, idle, no projects
Octopus autonomy	Assist, Execute, Autonomous
Plan preview	standard, every request, risky only
Capability policy	allow, ask, deny

Project permission modes	Accept edits (default unattended), plan (read-only) and others chosen per task
Meeting repeat	days (every N), weekdays, weeks (every N on chosen days), months (same date or nth weekday: 1–4 or -1 = last), optional end date
Billing types	fixed, hourly, retainer, none
Recipe target	any, all, a project
Schedule cadence	once, daily, weekdays, weekly
Contact preferred method	email, phone, text, whatsapp, none
Document filter types	All, PDF, Images, Text, Other
Module status	running, disabled, blocked, failed, invalid

3. Access, auth and global behavior

- **One shared secret.** Every API call sends `X-API-Key`. The page itself loads without auth but asks for the key on first visit and stores it in the browser's localStorage (`cq_api_key`). A wrong key shows "That key didn't work…" and clears it. Ten wrong keys from one address in a minute lock that address out (HTTP 429). The service refuses to start if HOST is not 127.0.0.1 and the key is empty.
- **No users and no roles.** The only "roles" are a contact's key/stakeholder link on a project. There is **no client portal**: clients and contacts are records. The only thing a client ever receives is a Google Calendar invite.
- **Network:** reachable over Tailscale; Web Push needs https (Tailscale Serve). Alexa needs a public path (Tailscale Funnel, only `POST /alexa`, verified by Amazon's signature).
- **Global scope switch:** Work / Personal / All, which filters by project category. On phones it appears on the Projects page; on desktop at the top of the sidebar.
- **Polling:** the dashboard refreshes every 5 seconds (skipped while the tab is hidden); a 1-second tick runs while anything is running, rate-limited or awaiting approval; a 30-second tick refreshes "2m ago" labels. Views patch the page in place so polling never interrupts typing.
- **Presence:** while the tab is visible and was touched in the last few minutes, the page sends a heartbeat about every 20 seconds. After 90 seconds without one the owner counts as away, so Alexa can announce an approval instead. Held in memory only.
- **Credentials are never displayed:** Claude tokens, OAuth tokens and API keys (only the last four characters of an API key) must never appear in the UI.

4. Processes (detailed workflows)

4.1 Task lifecycle (the heart of the app)

1. **Entry points:** the Ask box on Home; the New task sheet (⌘K, N or the + button); a recipe; a schedule; a backlog item; voice; Alexa; the personal assistant; a phase autopilot tick; SwiftBar / iOS Shortcuts through the API.
2. **Queue.** A new task goes after everything queued with the same or higher priority. The user can reorder by drag, move a task to the top, cancel it (with Undo), pause it (a queued task is skipped and keeps its place; a running Claude Code task is frozen and paused time does not count toward the timeout), play it, or stop it (kills the task and everything it launched).

3. **Run.** One task at a time per Claude account, so projects on different accounts run side by side. Each tool call becomes a live step in the task's log (for example "Running npm test"). The same stream reports real 5-hour and weekly usage.
4. **Rate limit.** The task shows "rate-limited" with a live countdown and resumes by itself at the reset time. "Retry now" is available. "Ping me when back" sends one push when a pause ends. "Save the session for urgent work" holds Low tasks while the 5-hour window is over 85%. A project pinned to a Claude account waits for that account and never moves to another.
5. **Sleep mode** ("Going to sleep" on Home): the queue keeps going and uses limits fully; Low tasks are no longer held back; a limit with no reset time is retried every 15 minutes; approvals don't ping the phone and auto-deny after 5 minutes (the task is told to carry on without it). It ends after N hours (default 8, max 24) or when the user presses "I'm awake".
6. **Approvals** (see 4.3) can pause a task mid-run.
7. **Finish:** *done* (with an answer); *needs_review* (the user presses "Looks good" to mark it done); *error* (Retry, Open chat or Dismiss); *cancelled*.
8. **Chat.** Every task is a conversation: all messages, Claude Code's step log, reply footers (status, model, time, cost) and a reply box. A reply resumes the same Claude session (`thread_id`). A reply can switch model (API models get the earlier messages, oldest dropped to fit the context window; Claude Code resumes its session plus a catch-up transcript). The choice sticks for later replies. A chat never changes kind midway.
9. **Other task actions:** edit, retry-with-edit, run options (model, mode, priority, when), summarise (what was asked, what got done, where it stands; cached until the thread changes), transfer chat, move to another project, bulk move/priority/re-run/cancel/delete, delete to the recycle bin and restore, mark read/unread, requeue (front or back), "Retry now".
10. **Task detail page:** plan steps, files changed, git branch and commits, tests seen, approvals, activity, result and the full diff.
11. **Titles:** a quick name cut from the prompt shows at once, then the cheapest answer-only model replaces it with a 2–6 word title.
12. **Spend rules:** "API spend" and budgets count real money only (OpenAI, Gemini). Claude Code cost is an estimate (a subscription does not bill it) and is shown on each task but not counted as spend. A project that reaches its monthly budget is paused and the owner gets a push.

4.2 Octopus: understand, plan, route, execute, validate, report

1. **Understand.** Instant commands first (no model): "what's happening?", "stop the current task", "cancel that" (a waiting plan first), "continue the previous task", "list my projects", "add X to the backlog of Y". Then personal assistant, image, then a cheap classifier labels the request coding / general / simple (keyword rules are the backup).
2. **Plan** (`octopus/plan.py`): project, agent (agents.json), model (scored), tools (tools.json), permissions, risk. Model choice weighs strength, cost, speed, track record and the user's configured order; scarce Claude Code usage is penalised for work that doesn't need it, more so when the 5-hour window is nearly used. Coding is non-transferable (it waits for Claude Code rather than hopping to a model with no project access). Every decision is stored with each candidate's score and shown on the Routing page.
3. **Review (optional).** A plan card (the OCTOPUS PLAN) shows **[Cancel] [Run]**. Run may change project, model or agent first and only ever queues once. A plan is shown when it is risky, blocked, needs an

answer, or the user chose "every request". "Don't change anything yet" in a request runs Claude Code in plan (read-only) mode.

4. **Execute** in the lane: agent, api (text or image), assistant, or an instant command.
5. **Validate:** tests seen in the run, files changed, git branch and diff.
6. **Report:** text on the dashboard and a `spoken` line for voice.

Settings that shape it

- **Autonomy:** *Assist* investigates and proposes (coding runs read-only); *Execute* runs normal actions and asks for risky ones; *Autonomous* also runs what would ask — but protected capabilities (deploy, force-push, delete, production, send email, admin) always ask unless the user sets them to Allow.
- **Plan preview:** standard, every request, or only risky ones.
- **Agents** (agents.json): instructions, preferred models, allowed tools and capabilities, context needs, execution policy (lane, when to show the plan, Claude Code mode per autonomy level, cost limit). Agents named in the product include the Creative Agent (images), Research (web-grounded), the Personal Assistant and the Voice Partner. **Tools** (tools.json): what a request needs; only those are given to a task (the rest are switched off for Claude Code); MCP servers can be added as "mcp" entries. Both are edited in Settings → Agents & tools and re-read on change with no restart.
- **Models** (models.json): provider, capabilities, ratings, context, price, optimize mode per category (cost / balanced / quality), ordered candidates per category, `vision` flag, `scarce` flag. Models can be switched on/off and added/edited/removed from the AI Providers page (openai, gemini, claude_code).
- **Pictures:** up to 6 per message (PNG/JPEG/GIF/WebP, 10 MB each). Routing picks only vision-capable models; Claude Code is told where the files are; API models receive real image parts.
- **Prompt polish:** a "Polish" button rewrites a prompt using the cheapest answer-only model.
- **Image generation** (gpt-image-1, gemini-image) saves pictures under workspace/images; **Research** uses web-search grounding.

4.3 Approval flow (Bash commands needing a human)

1. Claude wants to run a command that unattended mode does not cover. A permission-prompt server writes a pending approval and the task waits.
2. **Where the user is told:** a card in "Needs you" on Home; a toast and the notification panel; a phone push with **Approve / Deny / Open dashboard** buttons; the Dock/app badge count; and Alexa if the user has been away 90 seconds or more (not during sleep mode).
3. **The card shows:** the command, a heuristic risk read with chips, the steps the task took before asking, and the project's uncommitted git diff.
4. **Decisions:** *Approve once*; *Always allow* (adds the command to the project's allow list — an entry covers program plus subcommand, e.g. `npm test`; risky commands such as rm, sudo, curl, force-pushes and anything mentioning production are only allowed exactly as approved; commands that chain, pipe, redirect or substitute never match); *Deny* with an optional note that is passed back to Claude.
5. **Shortcuts:** swipe right to approve and left to deny (110px threshold); `A` and `D`; `Esc` closes. The approval pane has a sticky "Approve once" (52px), "Always allow" and "Deny with a note".
6. **Link approval (from a push).** The buttons are signed one-time links `/approvals/{id}/quick/{approve|deny}?t=<token>` (HMAC of the approval and the decision), because Safari cannot send the API key header. Tapping one opens a small standalone page with a

title, a text line and the command in a code block, supporting dark mode and an "Open dashboard" link:

   - **Approved ✓** (green accent): "Octopus is carrying on."
   - **Denied** (red accent): "Octopus won't run this and will work around it."
   - **Link not valid** (403): "This approval link is broken or was changed."
   - **Not found** (404): "That approval doesn't exist any more."
   - **Already decided:** says approved, denied, or closed (timed out or the task stopped).
7. The service worker's Approve/Deny notification buttons call the same URLs and then show an "Approved ✓" or "Denied" notification.
8. **Timeout:** unanswered approvals are denied after 1 hour (5 minutes in sleep mode) so the queue never stalls.
9. Assistant approvals (email, calendar) are separate cards that run exactly what was shown, once; approvals unanswered for 24 hours expire; message content is dropped from the database after the decision. The assistant itself can never answer approvals, only count them.

4.4 Phases and autopilot

1. The user splits a project into ordered phases (Phases tab) and puts to-dos in each; they can also move loose backlog items into a phase. AI task generation creates up to 15 to-dos for a phase using a chosen model.
2. **Start** makes a phase active and queues its first open to-do immediately. **Pause** stops feeding the phase while queued/running work carries on. **Complete** closes it whatever is left. **Reopen** sets it back to planned. Reorder phases by drag.
3. **One to-do at a time:** the next is queued only when the last finished. If one fails or needs review the phase stops and the owner gets a push — no more work piles on.
4. **What starts a phase** (project autopilot mode): *On my command* (only on Start, or by saying "start the next phase of KodeAura7"); *By the timeline* (a phase starts on its start date, in order); *When the limit allows* (phases run back to back while Claude usage has room).
5. **Usage ceiling:** default 85% of weekly usage (adjustable 10–100%) plus a fixed 90% cap on the 5-hour window. It holds automatic work until there is room; a phase the user starts by hand ignores it for its first item. The UI shows capacity/utilization.
6. Every task for the project also sees the phase plan and the current phase's goal.
7. The scheduler ticks every 30 seconds and returns pushes: started, done, blocked, late.
8. **Errors to design for:** "That phase is already done."; "There's nothing to do in this phase yet - add some todos to it first."; a CRM-only project (no folder) can only plan phases, so Start/autopilot show a "no folder" hint.
9. **Voice:** "start phase 2 of shop", "start the Build phase", "pause the phase of shop", "how are the phases going on shop".
10. **Progress report:** a Markdown report for the whole project or chosen phases, with an optional model-written overview (`POST /projects/{id}/report`).

4.5 Backlog (to-dos) flow

1. The Backlog is a per-project list separate from the queue. Add items by text (the composer says "Saving to backlog · won't run yet"), attach pictures, Polish text.

2. Drag to order; star important items; expand a row to set model, approval mode, priority (Low/Normal/Urgent) or edit text.
3. Tick items, then **Run selected**: they join the queue in the user's order (not selection order), each with its own model/mode/priority. The bar shows "N selected · joins after N tasks".
4. A **"Queue backlog in order"** button also lives on the Work tab's Octopus card. The Suggest button queues a read-only review of the project and proposes tasks. Dragging a ticked row moves all ticked rows; drag is disabled while a row is expanded.

4.6 CRM

- **Project record:** fields in section 2. Errors are written to be shown to the user as-is (for example due and completed dates cannot be before the start date). Computed **schedule** (days left, due state) and **progress** (to-dos done and total, phases done and total, percent; each to-do counts as one unit, cancelled excluded; a phase with no to-dos counts as one; a done phase counts all its to-dos as done; percent is null when there is nothing to measure).
- **Views of projects:** Cards, Table (sortable, includes a "Last activity" column) and Board (by stage; dragging/picking a stage moves the status).
- **CRM dashboard** (scope all/work/personal, optional client filter; excludes General and hidden projects; "open" = stage lead/proposal/active/on_hold and not completed/archived):
  - **Counts:** projects, open, overdue, due this week, due this month, at risk, follow-ups overdue, follow-ups today, follow-ups due, stale. Home's compact "CRM card" shows these.
  - **Lists of project rows:** overdue; due this week (0–7 days); due this month (8–30 days); at risk (health at_risk or off_track, off_track first). Each row: id, name, category, status, stage, priority, health, dates, contract value and currency, client name, color, image, owner, days left, due state, progress percent, last activity time.
  - **Follow-ups:** overdue and today.
  - **Pipeline:** one entry per stage with label, count and value per currency.
  - **By client:** active value per client ranked in the main currency (active projects, open projects, value per currency); projects with no client appear as "no client".
  - **Stale:** open projects with no activity for 14 or more days, with idle days.
  - **Workload:** an overdue bucket plus 8 weeks (week start/end, projects due, follow-up count, value).
  - **Currency:** a main currency is used for ranking and bar charts; amounts in different currencies are never added together.
- **Clients** have their own page: details, projects (stage, schedule, value, billing type, priority, health), contacts and activity feed. The sidebar lists clients with project counts, a "N past clients" link and "+ New client". The project record shows a client chip (id, name, status, color, logo) and its linked contacts with the key contact first.
- **Contacts** have their own page and feed. Contacts without an email cannot be invited to meetings (the UI must say which).
- **Activity feed** (`/crm/feed` for exactly one of project, client or contact): **open** follow-ups pinned on top (overdue first, then soonest), then **items** newest first mixing activities, done follow-ups and (for a project) finished tasks. A contact's feed includes every activity they took part in; a client's feed includes its projects' and contacts' activity. Limit 150. Forms to log a call, meeting, email, note or decision with summary, outcome and people involved.

- **Reports:** project report and client report, as JSON, CSV or Markdown (named like `slug-client-report-YYYY-MM-DD.md`). They include details, schedule or value, progress, contacts, stage history, activities and follow-ups; the client report also lists its projects with stage, due date, value and progress. Capped at 1,000 activities.
- **Pictures:** project and client pictures (up to 2 MB, type checked from file contents, shrunk to a small square by the dashboard before upload) replace the colored letter tile everywhere; a separate tile color sits behind it.
- **Hidden projects** appear under "Hidden"; deleting a project deletes its data but leaves the folder on disk untouched.

4.7 Follow-ups and reminders

1. Add a follow-up to a project, client or contact with a date and optional time.
2. At the due time (or 09:00 local if no time) a push goes out titled "Follow-up · <subject name>" with the message "<title> - due at 14:30 / due today / was due <day>". Tapping it opens that project overview, client or contact.
3. A follow-up added or re-dated into the past is marked as already reminded (no instant push). Re-dating or reopening restarts its reminder.
4. Follow-up reminders are held through quiet hours (23:00–08:00).
5. Marking done stamps the time; reopening clears it. Done follow-ups appear struck through in the Calendar for 30 days.

4.8 Timeline, calendar and meetings

Kind	Can go overdue?	Default alerts (days ahead)
deadline	yes	7, 3, 1, 0
milestone	yes	3, 1, 0
task	yes	1, 0
presentation	no (simply over)	3, 1, 0
demo	no	3, 1, 0
meeting	no	1, 0
event	no	1, 0

1. **Alerts:** at each alert day a push such as "Deadline tomorrow · <project>" with "'Title' with <people> - <when>." For due kinds, presentations and demos it also lists the outstanding open work. A timed item that isn't a due kind also gets a push **60 minutes before** ("<Kind> in 60 min · <project>"). When an item goes overdue, one push "Overdue · <project>". Alerts already inside the window at creation are marked sent; a missed earlier alert is never sent late. Tapping opens the project timeline.
2. **Timeline health** per project: on track, at risk (a due item, presentation or demo within 3 days while work is still open), overdue, done.
3. **Meetings** add contacts, extra guest emails (≤50), duration, repeat rules, and a **meeting notes page** that saves as you type (notes are never sent in invites). A repeating meeting stays one row; its date shows the next occurrence; alerts are tracked per day.
4. **Calendar screen:** month and schedule view of project dates, follow-ups, timeline items and personal Google Calendar events. The same Calendar is a tab in the desktop right rail and the phone right

drawer.

4.9 Google Calendar invites (the only feature that emails outside people)

1. **Requires** a connected Google account with Calendar (Personal → Connections). Without it the API returns 422: "Connect your Google account (with Calendar)…".
2. In the meeting form the owner turns on **Send invite** and optionally **Meet link**. Only meetings can send invites.
3. The Den saves the meeting, then creates the Google event on the primary calendar with all guests notified. Guests = contacts' CRM emails + extra guests. The meeting keeps the event id, invite link, Meet link and the Google account used.
4. Google emails each guest the usual invitation with Accept and Decline. The repeat rule is translated to a Google recurrence rule.
5. **Editing** (title, date, time, duration, guests, contacts, repeat) patches the event; guests get Google's update email; replies are preserved for guests who stay.
6. **Cancelling:** the invite is cancelled (guests told) when "Send invite" is turned off, the meeting is cancelled, its kind changes away from meeting, or it is deleted. `DELETE` returns invite_cancelled, or a warning if Google cancel failed; the meeting is deleted anyway.
7. **Failures never lose the meeting.** It is saved with an **invite error** (≤500 characters) shown under it; saving again retries. If the event was deleted in Google the error reads: 'The invite was deleted in Google Calendar. Turn on "Send invite" and save to send a new one.'
8. **UI states:** none; sent; sent with a Meet link; error; contacts with no email.

4.10 Morning briefing

- A "Good morning" page: greeting, date, today, overdue, upcoming (7 days), today's meetings (Google Calendar, if connected), approvals waiting, attention (failed or needs-review tasks from the last 7 days), reminders, queue counts, at-risk projects, plus a headline and text/spoken renderings.
- Buttons: **Read it to me**, **Refresh**, **Calendar**. Shortcut `G`.
- Pushed once a day at `briefing_time` (default 08:30) when `briefing_push` is on; tapping opens the briefing.
- A built-in **"Good morning"** Quick button on Home runs it.

4.11 Notifications

Three channels

1. **ActivitySmith** phone push. Levels vary by kind; urgent kinds (approval, failure, reminder) get high urgency and a 1-hour lifetime, others 24 hours. A push has a title, body (≤600 characters), kind, deep-link and optional approval id. Approvals and failures are time-sensitive (break through Focus); completions are a normal banner.
2. **Web Push** to the dashboard installed as an app (Safari › Add to Dock, or iPhone Home Screen). Needs https. The VAPID key pair is created on first use. Subscriptions are per browser; a gone subscription is removed automatically. Approvals stay on screen until acted on; the Dock icon shows a red count of approvals waiting. A web app cannot bounce its Dock icon.
3. **Alexa:** if an approval is waiting (or a task started from Alexa finishes) and the user has not used the dashboard for 90 seconds, the Echo lights up yellow. Only a count goes through Amazon's service;

details are read by the skill. Nothing is sent while the user is on the dashboard, mid-conversation with the skill, or in sleep mode.

Push events

- Approval needed; task failed; task done; message answered; rate-limit ended ("Ping me when back"); budget reached (project paused); follow-up due; timeline alerts (ahead, 60 minutes before, overdue); morning briefing; phase started, done, blocked or late; assistant reminders.
- **Per-project toggles:** approvals, failures, completions, quiet hours. **Quiet hours** 23:00–08:00 hold everything except approvals. A "Send test" push is available.

In-app notification center

- A bell in the topbar with an unread badge (capped "9+"), toasts, and a panel dialog.
- **Kinds:** done, answer, approval, approved, stop, deleted, restored, queue, error, review, limit, weekly, add; polish and info are toast-only. Approval, error and review count as "needs you" and their rows carry the CTAs "Review request", "See what broke" and "Review changes".
- **Toasts:** top-right, stacked newest at the bottom, with kicker, title, sub-line, optional action (e.g. Undo), a ✕ and a timer bar. Tapping a toast carrying a task opens that task. Pending approvals show Deny and Approve buttons right in the toast and the panel row. No toast appears if the related chat is already open.
- **Panel:** header "N new", "Mark all read" (desktop) / "Read all" (phone), ✕; tabs **All** and **Needs you**; grouped rows with an unread dot; footer with count, "Allow desktop alerts" and "Clear read". Closing marks everything read. Empty: "All caught up" / "Nothing waiting on you". First load backfills the last 24 hours plus anything still waiting; a weekly-limit early warning is computed server-side.
- **System notifications** need a secure context; over plain http (Tailscale) the app relies on ActivitySmith. Over https, allowing them also subscribes to Web Push and no duplicates show.
- **Status banners** (sticky strip under the topbar): "This device is offline"; "Can't reach Octopus" (after 2 failed polls); "Octopus is restarting" (with "Retry now"); "Rate-limited" (how many tasks are paused and the auto-resume time); "Sleep mode on" (with "I'm awake").

Email notifications (to the owner only)

- Sent through the connected Google account's Gmail (needs the gmail.compose scope) with subject prefix "[The Den]". No SMTP. Sending never raises, runs on a background thread, and does nothing silently if Google is not connected. Recipient: setting `email_to`. Per-event switches: `email_events_off`.
- Every email uses a fixed template (no model call): a colored header bar, a body of paragraphs or bullets, and a footer "The Den". Header colors: info = blue, good = green, warn = amber, bad = red.

Event key	Kind	Trigger and content
restart_needed	warn	Source code changed after start; sent once per batch after 10 minutes of quiet; lists changed files; says to restart from Settings → Restart
restart_requested	info	The owner asks for a restart; mode (normal or hard) and detail, e.g. "Waiting for N running task(s)…"
restart_completed	good	Next start after a requested restart; when asked, start time, how many held tasks were released
unexpected_restart	bad	A start with no restart requested (crash or reboot); running tasks marked interrupted; Retry continues them

project_created	good	Name, category, folder (or "none (CRM-only)")
project_deleted	bad	Data deleted; folder on disk untouched
project_archived	warn	Can be moved back any time
project_completed	good	"<name> was marked Completed."
test	info	"Email is working" (Send test button; 409 if Google is not connected, 502 if Google refuses)

Settings screen needs: a connected / not-connected indicator, the recipient, 9 toggles (unknown keys are rejected with 400), and a Send test button.

4.12 Personal assistant (Email, Calendar, Reminders, Connections)

- Talk to Octopus in plain language: read the calendar, search and draft email, schedule and move meetings, set reminders, look at the queue and projects, hand work to the app's AI models.
- **Flow:** intent rules (no model) → assistant lane → plain tool calls when the words are enough ("what's on my calendar tomorrow", "latest email from X", "block 3 PM tomorrow") with no model, otherwise a planner model that only proposes tool calls as JSON (falls back to the local claude CLI with every tool off) → the single gate (capability policy allow/ask/deny, approval, audit log) → scope check (agents.json + tools.json bind per call) → tools → providers (Gmail, Google Calendar; Outlook not built).
- **Tools:** email, calendar, reminders, agent (hand work to Claude Code), workspace (queue, tasks, task, projects, backlog, usage, add_todo, cancel_task), ai (models, ask, image).
- **Conversation memory** per chat so "which one is at 3 PM?" → "move it to 4" works (the move still asks first).
- **Permissions** (Personal → Connections → Permissions):

Capability	Default	Capability	Default
EMAIL_READ	allow	CALENDAR_READ	allow
EMAIL_DRAFT	allow	CALENDAR_CREATE (nobody invited)	allow
EMAIL_SEND (send/reply/forward)	**ask**	CALENDAR_UPDATE	**ask**
EMAIL_DELETE	**deny** (no such action exists)	CALENDAR_DELETE / CALENDAR_INVITE	**ask**
REMINDER_*	allow	AGENT_CODING	**ask**
PROJECT_READ	allow	TODO_WRITE	allow
TASK_CONTROL	allow	AI_MODEL_USE	allow
WEB_ACCESS	allow	IMAGE_GENERATE	allow

  A call needing several capabilities gets the strictest. Setting a normally-ask capability to allow is "trusted" and the activity log marks it so.
- **Safety:** once a run has read text written by someone else (an email body, an invite, a task, backlog item or another model's answer), outbound actions ask anyway (a prompt-injected email cannot send mail). In Assist autonomy, anything that changes something asks.
- **Approval card** shows exactly what will run (recipient, subject, message; or title, date, time, attendees, calendar and which Google account). It runs once from the stored arguments, never regenerated.

- **What is deliberately not possible:** the assistant cannot answer approvals; only active projects are visible (an archived or budget-paused project is "can't find it"); personal-assistant chats are listed by status only, never their text; pending-approval details are not shown, only their count.
- **Connections page:** NOT CONFIGURED state with setup steps until Google keys exist; Connect Google (OAuth opens on the Mac); "+ Add another Google account"; each account's row sets its default for email and for calendar; "Reconnect needed" when a token expires (apps in Google "Testing" expire after 7 days); revoke/disconnect. With several accounts Octopus reads all of them, replies from the account that has the message, and sends new items from the default account; the user can say which account to use.
- **Personal pages:** Email overview, Calendar overview, Reminders (list and cancel), Connections (accounts, permissions, activity log). Activity log never holds bodies or tokens.
- **Limits:** forwarding doesn't include attachments; free-slot search only sees attendees whose calendars are shared; reminders push through ActivitySmith only.

4.13 Voice and Alexa

- **Voice** is a way in and out of the same pipeline, never a second assistant. Mic button in the Ask box and a floating mic button for a voice chat; the Voice page holds audio options.
- **Providers:** speech-to-text = the browser's recognition or local Whisper (not installed by default); text-to-speech = the browser's voices or macOS `say` (can speak on the Mac).
- **Settings:** speaking speed (0.5–2), default voice, a voice per language (e.g. Hindi → Lekha), automatic language detection (writing system plus common words), and a manual voice (from the speaker button in the Ask box) that overrides everything until "Automatic". macOS voices are a server setting; browser voices are per device.
- Follow-ups attach to the last chat; a spoken plan is answered "yes"/"no"; `/voice/command` returns the spoken line. Shortcuts clients poll `/octopus/result/{task_id}?wait=30`.
- **Alexa** ("Alexa, open my octopus"): reports the queue and what Octopus is working on; reads approvals in plain words with risk, then approve/deny/skip; "add a new task" (project, prompt, read back, "yes"); "push the current code to GitHub" (never force-push); "let's discuss a task for KodeAura7" (a real back-and-forth with the Voice Partner agent, "queue it" hands the conversation to Claude Code); "stop the current task" (asks first); "go on" for a late answer; anything else goes to Octopus. A conversation survives session timeout for 15 minutes. An optional spoken code is required for voice approvals. The discussion is saved as one chat.
- **Voice page, Alexa section** shows: skill ID set, paired, notifications allowed, public URL, last error, a Send test notification button, and the code setting.

4.14 Accounts, usage and money

- **Claude accounts** (not user logins): this Mac's own login (can be hidden once a token account exists) and extra token accounts (from `claude setup-token`). Each shows label, email, alias, color, org/plan, logged-in, active (default), 5-hour and weekly usage, limited-until, last hit, auth error and the projects that pin it. Auto palette: #e8825a #4f9de8 #4fbf8a #b583e8 #e8c24f #e8627f #4fc6c9 #9aa3b2.
- **Actions:** add (token verified with one tiny request, a few seconds, before saving), delete, set alias, set color, edit details, show the Mac account, make default (activate), refresh usage. The account switcher sits in the topbar; the project settings and New task sheet also have an account field.

- **Usage pill** (topbar): two rows (5h and weekly) with bars, percentages and reset times. Color: teal below 80%, amber from 80%, violet from 95%. On phone project pages it is a 36px ring showing the 5h percentage. Tapping opens a usage sheet.
- **Token usage page:** a local date range (default 30 days), totals versus the previous period, per day, per project / model / lane (agent, api, assistant, background) / category, and every run behind the totals. Background = side calls such as titles, summaries, the classifier and prompt polish.
- **Usage summary:** real API spend (today, month, last 7 days, by project) and Claude's 5-hour / weekly usage.
- **API keys** (OpenAI, Gemini; AI Providers page): several per provider with a default; on rate limit, no credit or a rejected key the call falls back to the next key (a rejected key is marked). Add flow checks the key with one free request; only the last four characters are shown afterwards; the environment key is listed first and cannot be removed there; a pasted key works without a restart.
- **Budget:** per-project monthly API cap (default $20, +/- $5 stepper in project settings) and a global monthly budget; reaching it pauses the project and pushes.

4.15 Notes, documents, pictures, recipes, schedules

- **Notes pad:** list of notes (title, body, project, meeting link); save as a Markdown file into a project's folder, the shared notes folder or a folder under the home directory. A project with no folder gets an error rather than silently using the shared folder. A note tied to a meeting survives meeting deletion.
- **Project notes tab:** add form with kind chips (Guideline, Requirement, Note), picture attach, Polish and Add; edit buttons; a "Meeting notes" section from calendar items.
- **Documents tab:** add files (≤25 MB each), refresh, search, a List / Compact / Thumbnails switch, a sort select, type chips (All, PDF, Images, Text, Other); per file preview, rename, download and delete; skeleton while loading, "No documents yet" or "No documents match." when empty.
- **Recipes:** saved prompts with Run (in one project, or in every active project for an "all" recipe). On a project, "Run here". Recipes and schedules still queue through the older router path.
- **Schedules:** checked every 30 seconds; one that was due while the Mac slept runs once on wake. Rows have an enable toggle; add, edit, delete. Pictures on scheduled prompts are not supported.
- **Quick buttons:** one-tap prompts on Home; an Edit/Done toggle removes buttons; "+ Add".
- **Recycle bin / Hidden:** deleted tasks and hidden projects with restore.
- **Learn a website** (Settings → Sites).

4.16 Backup, restore, restart, modules

- **Export** tasks, history or settings as json or csv (anything that looks like a credential is stripped). **Backup** downloads the whole database (private).
- **Restore:** upload a file (≤512 MB) → *check* (inspects it; requires the projects, tasks and settings tables) → *confirm*. The current data is first saved to `queue.db.bak-pre-restore` so a wrong restore can be undone. The UI needs a strong warning and a check-then-confirm flow.
- **Restart Octopus** (Settings or the topbar): normal (waits for running tasks) or hard; native confirm. The page polls the service every 2 seconds and reloads when the process id changes. The offline popup is never shown during a restart. Email events cover the request and the result.
- **Modules:** drop-in features in `modules/<name>/` (manifest keys: name, version, description, enabled, entry, requires/optional, config with typed keys, secrets, capabilities, hot_reload). Lifecycle: discover, validate, load, initialize, start, stop, unload, reconciled in dependency order. A module can register

tools, commands and task-added / task-completed listeners. Module tools still need an explicit grant in tools.json and the agent's tool list. Only example today: `modules/hello`. API: list, rescan, commands, enable, disable, reload, config. **No dashboard screen exists yet**; the docs note a command-line input in the dashboard is an unbuilt UI task.

5. The current UI

5.1 Architecture

- `dashboard.html` is a shell: `<div id="app">`, about 22 CSS files and `app.js`. Rendering builds HTML strings and patches the DOM in place (so polling never interrupts typing); events use `data-action` delegation. No build step.
- Meta: viewport-fit=cover, interactive-widget=resizes-content, theme-color #0B1016, iOS standalone web app, manifest, title "Tushar's Workspace".
- **Locked gate:** "Workspace is locked" with a sleeping Octopus, an X-API-Key password field and an Unlock button.
- **Hash routes:** home, projects, recipes, activity, personal (email, calendar, reminders, connections), messages, routing, providers, settings, voice, usage, calendar, briefing, bin, hidden, notes, clients, crm; plus `#/project/<slug>/<tab>` (old `#/project/12` links still resolve), `#/client/<id>`, `#/contact/<id>`, `#/task/<id>` (opens Home with that chat). `?status=done|error|needs_review|cancelled` (from SwiftBar) opens Activity on that filter. Every view change shows a 520 ms skeleton.
- **Service worker** (`sw.js`): Web Push only, caches nothing; approval pushes use requireInteraction with Approve/Deny action buttons; sets the app badge from the pending count; clicking a notification focuses or navigates an open dashboard window, or opens one.
- **Splash on every load:** a dancing Octopus, a greeting ("Good morning, Tushar!"), a recap pill ("while you were away: N done · M need you", or a first-run line) and "stretching tentacles…". Dismissed only by a double tap or double click (380 ms window). Records last-seen in localStorage.

5.2 Desktop navigation (900px and up)

- **Sidebar** 250px (resizable 200–420px, drag handle, double-click resets). From 1280px it sits inline and collapses to a 60px icon rail with `[` or ⌘B; from 900 to 1279px it is a slide-over drawer with a scrim and the rail underneath.
- **Sidebar contents:** brand (Octopus in its current mood, "TUSHAR'S / Workspace") and collapse toggle; a status pill (pulsing dot plus the hero's short text) linking to Home; the Work / Personal / All switch; nav items **Den Home** (key 1, with a needs-you badge), **Projects** (2), **Clients**, **CRM**, **Recipes & schedules** (3), **Activity** (4), **Notes**, **Recycle bin**, and **Hidden** (phone only, with a count badge).
  - **Projects** expands inline: groups General, Work, Personal; each row has a swatch, name, CRM badge, due badge and a meta word (needs you, running, asleep, failed, N queued, paused); ends with "+ New project".
  - **Clients** expands to one row per client with its project count, a "N past clients" link and "+ New client".
  - **Activity** expands to the filters All, Done, Failed, Review, Cancelled, each with a count.

  Then the **PERSONAL ASSISTANT** group (Email, Calendar, Reminders, Connections, with pending-approval counts) and the **OCTOPUS** group (Token usage, AI Providers, Routing, Voice, Settings),

"Keyboard shortcuts · ?", and a footer with a coral **New task ⌘K** button.

- **Collapsed rail:** icons for each nav view, Personal, Notes, Settings and Bin; a separator; one swatch per project (General shows the Octopus); a "+" new-task button.
- **Topbar:** title; a search box (Spotlight-style results); live/offline indicator; Claude account switcher; usage pill; restart-Octopus button; the notification bell. The status banner strip sits under it.
- **Right-edge rail** with rotated tabs: **Queue** (badge = queued + running), **Messages** (amber badge = chats waiting on you), **Calendar**. Several can be open side by side, each resizable and stackable; tabs are draggable to reorder and a panel header can be dragged to a new column. `⌘⇧B` cycles Queue → Messages → Calendar → closed.
- **Chat pane:** docks inside the Queue or Messages sidebar, floats, minimises to a bottom-right chip with a tab count, or sits as an inline column (resizable, 360px min, 460px default). Chat tabs sit above it; each shows a live dot, title, "#id · project", a pin button and a close button; "+" starts a new chat; an icon returns to the message list or queue. A tab right-click menu offers Pin, Close and others. A mic FAB (bottom right) starts a voice chat.
- **Pinning today:** only *chat* tabs in the desktop pane can be pinned (stored in localStorage `octopus-pinned-tabs`). Pinned chats list at the top of Messages under "PINNED" (desktop only, when no search or filter is active). Close others / Close all keep pinned tabs; closing a chat unpins it.

5.3 Phone navigation (below 900px)

- **Topbar:** left a menu button (opens the sidebar drawer) or a back arrow on project, client and contact pages; centre an eyebrow ("TUSHAR'S" on Home, "WORKSPACE" elsewhere) and the title; right: live dot (hidden below 360px), account switcher, usage pill (a 36px ring on project pages), search (takes over the whole header while open), refresh and the bell.
- **Bottom tab bar** (7 columns, in order): Home, Projects, Recipes, a coral **+** FAB (56px, new task), Activity, Personal, Messages. Badges: Home shows the needs-you count; Personal shows pending assistant approvals; Projects is highlighted on project pages. It steps aside while the keyboard is open.
- **Left drawer** (the sidebar): opens from the menu button or by swiping right from the first 24px of the screen; closes by swiping left or tapping the scrim.
- **Right drawer** (Queue / Messages / Calendar): opens from a thin grip on the right edge (tap, or swipe left) with a segmented switch with badges; closes with the scrim or a swipe right.
- **Page swipe:** a 3D carousel across Home, Projects, Recipes, Activity, Personal, Messages (axis lock |dx| > 1.3|dy|, commit at 70px, edge resistance 0.25). A swipe right on a project, client or contact page goes back. Ignored in the first 24px (drawer) and inside `data-noswipe` areas such as chip rows and tab strips.

5.4 Screens and what each holds

Splash, lock and onboarding

- Locked gate (5.1), splash (5.1), and the first-run onboarding "Give me something to hold" (no projects) with add-project steps and a first-run tour.

Home (Den Home; phone title "Workspace")

- Two columns on desktop (from 700px), Ask box docked sticky at the bottom (above the tab bar on phone).
- **Left column in order:**

    - **Hero:** mood Octopus, a status label with a pulsing dot, the status line, a sub-line (hidden in compact density) and action buttons. States: waiting, rate-limited, working, held, idle, no projects.
    - **Octopus panel:** voice and agent pickers; the plans waiting on the user; provider availability; the autonomy switch; the "Going to sleep" button.
    - **Rollups:** Total, In queue, Succeeded, Failed. Each cell links to Activity or Projects with the matching filter.
    - **Needs you:** approval cards (command block, risk chips, Details, Deny, Approve; hint "A" on desktop, "or swipe →" on phone), failed cards (Retry, Open chat, Dismiss), review cards (Open, "Looks good"), plan cards (Cancel, Run).
    - **Rate-limited card:** live countdown, "Retry now", "Open chat".
    - **Queue tabs:** *Running now* (live step, progress), *Up next* (drag handles, move-to-top ↑, cancel × with Undo, bulk bar) and *Completed*.
- **Right column in order:** CRM card (the counts), Coming up, Follow-ups, Scheduled (rows with enable toggle, "Manage" link), Quick buttons, Usage (session and weekly bars, spend, 7-day chart), Models (status dot, name, usage, status line).
- **Ask box:** on phone a one-line bar that expands into a full overlay. Controls: auto-growing textarea (Enter sends, Shift+Enter adds a line), model picker, Claude account picker, agent override, plan preview, attach picture (also by paste, up to 6, thumbnails with ×), Polish text, a project target menu, microphone, and a send "↑". States: sending, a routed-result line (with an "open #id" link), error text ("Could not send: …").

Projects

- A **Cards / Table / Board** switch (CRM views); the Work/Personal/All switch (phone only); filter chips **All, Pending, Ongoing, In queue, Failed, Done** with counts; a toolbar with **Sort** (Most active, Recently used, Most queued, Name A–Z), **Filter** (Show paused projects, Only projects with tasks), Search and "+ New project"; a bulk bar when the In queue filter is on.
- Rows are grouped by section headings; each has a name, status and an expand toggle that lists its tasks. Two columns of rows from 900px.
- Empty states: "No projects yet" with "Create your first project"; "No projects match" with "Clear filters".

Project page

- **Header:** avatar (image, initial, or Octopus for General), name, a status dropdown (Not started, In progress, Need info, On hold, Completed, Archived), path, a Work/Personal switch, and buttons **Suggest**, **Hide**, **Pause/Resume**, **+ Task**. A CRM-only project shows a "CRM only" badge and "Link a folder". A paused banner has a sleeping Octopus. General has no status pill and no Hide or Suggest.
- **Tabs in order:** Overview, Work, Backlog, Phases, Timeline, Notes, Documents, Activity, Recipes, then a spacer and a Settings button with an icon. Counts show on Backlog, Phases (done/total), Notes, Documents, Recipes. The tab is kept in the URL. Overview is hidden for General. A CRM-only project opens on Overview and tabs that need a folder show a "no folder" card with "Link a folder".
- **Overview** (CRM record), **Phases** and **Timeline** contain the CRM record editor, phase planner and timeline/calendar content described in sections 4.4, 4.6 and 4.8.
- **Work tab:** main column with the ask box (sends to this project), Live cards (status label, title, progress bar, meta), Up next ("#position", title, chip) and Done recently (with an "All activity →" link); side column with stats (Backlog, Up next, Running, Done this week), the pinned note ("Octopus reads this before every run"), a 7-day chart with success rate and average run time, a monthly spend bar, a Phases card,

and the Octopus card with the "Queue backlog in order" button. Empty: "Nothing queued for this project" with "Queue backlog (N)" and "Suggest tasks". Collapses to one column at 1100px.

- **Backlog tab:** see 4.5. Header actions "Select all / Clear" and "Queue…"; sticky bulk bar with Delete, Run selected… and Select all; composer at the bottom with a model menu, a mode segmented control (Accept edits and others), attach picture, Polish and send.
- **Notes tab / Documents tab:** see 4.15.
- **Activity tab:** per-task rows with a select checkbox and a ⋯ menu (move or delete); bulk bar with Move…, Priority…, Re-run, Cancel, Delete, Select all / Clear. Empty: "No activity yet" with "Add a task".
- **Recipes tab:** recipe cards with "Run here". Empty: "No recipes yet".
- **Settings tab:** *How tasks run* (Claude account card, default priority, "Mark in progress automatically"); *Notifications* (Approvals, Failures, Completions, Quiet hours); *Permissions and deploy* (allowed-command chips with ×, deploy targets: name, how, "Add target"); *Autopilot* (mode and ceiling); *Spending* (monthly budget stepper and a bar); "Loading settings…" while loading.
- **Pinning requirement (new):** the project notes say **any tab on the project page can be pinned**. Today project tabs are **not** pinnable — this must be designed (see 8).

Clients, Client page, Contact page, CRM

- **Clients:** list with logo, name, status, project count, active value; "+ New client", past-clients link.
- **Client page / Contact page:** details, linked projects or contacts, activity feed with log forms and open follow-ups, report download (CSV / Markdown).
- **CRM:** a Cards / Table / Board switch and the dashboard tiles and lists of 4.6 (counts, overdue, due this week, due this month, at risk, follow-ups, pipeline, by client, stale, 8-week workload).

Recipes & schedules

- An intro line, a Schedules list with toggles, and a recipe-card grid. Empty states with "Schedule a task" and "Add a task".

Activity

- The queue section (same rows and controls as Up next), then History with filter chips (All, Done, Failed, Review, Cancelled), grouped TODAY / YESTERDAY / EARLIER, and a "Recycle bin" link. Empty: "No activity yet". An SwiftBar link can open it pre-filtered.

Messages (message centre)

- Every chat, one entry per task with its replies folded in, newest message first, grouped by day, with a PINNED section on desktop. Search matches title, project and every message (all words must appear) and shows the matching message; it searches the whole history. Filters: status (needs you / active / done / failed), project, model, type (Tasks / Messages), unread. Paged (40 at first, then "load more"). Key `6`.

Chat pane

- Header with title and a sub-line (project, model, status); a thread of You and Octopus bubbles, a tool/step log, inline approval cards and system pills; reply footers (status, model, time, cost); a "New reply" jump button when scrolled up; a typing indicator (dancing Octopus and a progress bar for agent chats); it auto-scrolls so a new long reply starts from its top. Composer: attach picture, Polish, a model and account menu, send. Phone header has a back arrow.

Calendar, Briefing, Notes, Bin, Hidden

- Calendar ("Loading the calendar…" spinner while loading); Briefing "Good morning" with "Putting your day together…" placeholder; Notes pad; Recycle bin; Hidden projects with a count badge.

Personal

- Tabs Email, Calendar, Reminders, Connections (section 4.12).

Routing, AI Providers, Token usage, Voice

- **Routing:** "Which model for what"; every routed request: classification, project, agent, model, tools, reason, outcome, duration, cost, each candidate's score.
- **AI Providers:** models grouped by provider with availability, limits and usage; on/off switch, add/edit/remove models; API keys cards.
- **Token usage:** section 4.14.
- **Voice:** providers, speed, per-language voices, Alexa section.

Settings (segmented tabs)

Tab	Content
General	Claude accounts; Tour and help; email notification settings; global toggles (notify on resume, hold low when busy, monthly budget); sleep mode state
Octopus	Autonomy, Plan preview, Permissions
Sites	"Learn a website"
Agents & tools	Edits agents.json and tools.json
Calendar & voice	Calendar and briefing settings (briefing time and push), voice
Shortcuts	Rebindable shortcuts with Reset all
Backup	Export and restore

Restart Octopus and the Modules registry are service-level controls; Modules has no screen yet.

5.5 Sheets, modals and overlays

- Bottom sheets on phone become a centred modal on desktop (640px wide).
- **New task sheet (palette):** textarea with a mood Octopus; project chips (General first); model chips (Auto plus each model); a Claude account field; an APPROVAL mode chip row (hidden for API models or scheduled runs); priority (Low/Normal/Urgent); when chips (Now, Tonight, Tomorrow, daily, weekdays and others); RECIPES and RECENT PROMPTS lists; footer with attach picture, dictate, Polish, an ETA line, a "Keep" checkbox and the "Queue ↵" or "Schedule ↵" button (spinners "Routing…", "Scheduling…").
- **Other sheets:** usage, account, accountEdit, addProject, keys (shortcuts list), summary, editTask, retryEdit, runOpts, chatActions, bulk, taskDetail, editTodo, editNote, editSchedule, quick (add quick button), runReview, event (timeline item / meeting), phase, report, genTasks, tour, help, preview, plan.
- **Spotlight:** Alt+Space, a search or ask-Octopus-anything overlay. **Search dropdown:** from the topbar box with keyboard selection. **Palette** has a dictate (mic) button.
- **Offline popup:** "CONNECTION LOST / Octopus is offline", shown after two consecutive failed polls and never during a restart; buttons Dismiss and "Try now"; includes a `launchctl list | grep claudequeue` hint and the last error.

- **Confirmation popover** anchored to a button; menus with a scrim (sort, filter, model, account, reply menu, link menu); tab menu; native `confirm()` for restart and hard restart.
- **Esc closes** in this order: link or tab menu, spotlight, search, notifications, menus, sheet, nav drawer, overview edit, ask overlay, right drawer, then the pane.

5.6 Keyboard shortcuts

Rebindable (Settings → Shortcuts, stored in localStorage `octopus-shortcuts`)	**Fixed**
New task `Mod+/`; Collapse sidebar `Mod+B`; Cycle right panels `Mod+Shift+B`; Spotlight `Alt+Space`; Search `/`; Approve `A`; Deny `D`; Briefing `G`; Shortcuts list `?`	`Mod+K` and `N`: new task; `[`: sidebar; `1`–`6`: Home, Projects, Recipes, Activity, Personal, Messages; `7`: Calendar; `Esc`: close

A new binding steals a combo from another action; built-in combos are refused. Letter keys are ignored while typing; only Mod combos work then.

5.7 Gestures and touch

- Page swipe carousel; swipe-back on detail pages; swipe to approve/deny on approval cards (`data-swipe`); drag reorder on any `.handle` in Up next and Backlog (a ticked row drags all ticked rows); edge swipes for drawers; double-tap to dismiss the splash; press-and-hold (500 ms) on any element with a title shows the title as a toast on touch screens without hover; auto-growing textareas.
- Rendering is held while a finger is on a handle or swipe card. Desktop resizers: sidebar, chat pane and right panels; double-click resets the width; widths stored in localStorage.

5.8 Responsive behavior

- Desk threshold 900px. Sidebar inline from 1280px, slide-over 900–1279px, drawer on phones. Chat pane inline column from 1280px, floating below. Home two columns from 700px. Projects two columns of rows from 900px. Project Work collapses at 1100px. Settings, CRM, clients and calendar have breakpoints at 520, 600, 700, 760, 860, 900, 1080 and 1100px. The topbar live dot hides below 360px.
- **Mobile rules (<900px):** every text input is 16px so iOS does not zoom; tap targets 40–44px; diffs scroll sideways; the tab bar uses a 7-column grid with a 56px FAB gap and is tighter below 360px; the bulk bar sits above the tab bar.
- The visual viewport shrinks the app height while the keyboard is open (`--app-h`). Safe-area insets are used. Reduced motion is honoured.
- **Density:** a compact mode (`data-density="compact"`, rows 44px, smaller hero, hidden meta lines; persisted as `octopus-compact`). Where the toggle is rendered in the current topbar was not found; check `toggleDensity` in app.js.

6. Visual system

- **Theme:** dark, with coral as the brand color. Dense, mono-labelled, card-based UI. Color is reserved for status. The Octopus mascot (code-drawn SVG, six moods) is the brand mark and appears in empty and error states. Icons are inline SVG; emoji appear only in Ask placeholders and Quick buttons. Copy comes in a playful tone and a plain tone.
- **Fonts:** Bricolage Grotesque 400–800 (UI) and JetBrains Mono 400–600 (numbers, meta, code), both from Google Fonts.

Type token	Size	Type token	Size
eyebrow	10.5px mono uppercase	row	14.5px
meta	11.5px	section	15px
small	12.5px	input	16px
body	14px	card title	18px
title	20px (22 desktop)	hero / display	23px / 31px

Reading line height 1.62.

Surfaces	Value	Borders	Value	Text	Value
bg-deep	#06090D	line-soft	#1A2330	text	#E8EEF3
bg	#0B1016	line	#223040	text-2	#DCE4EB
bg-sidebar	#090D12	line-strong	#2A3747	text-3	#C2CDD8
bg-pane	#0A0E13	line-btn	#33404F	text-muted	#A3B0BE
bg-sheet	#0F151D	line-focus	#3A4B60	text-meta	#93A1B0
inset	#0F141B	line-menu	#2F3D4F	text-faint	#6B7A8A
surface	#121922			text-ghost	#5B6A7A
surface-2	#18212C				
surface-3	#223040				
surface-drag	#1E2936				

Brand and status	Value	Used for
coral	#FF7B5C (hover #FFA48C, on-coral #1A0A05, tint 12% alpha, line 38% alpha, you-bubble #2A1A15)	brand, primary actions, New task, FAB
wait (amber)	#FFB547	waiting, needs you, 80%+ usage
run (teal)	#45E0C8 (soft #9FEFE2)	running, normal usage
err	#FF5470 (soft #FF8FA2)	errors, deny
review (blue)	#7FB2FF	needs review
sleep / rate-limited (violet)	#A58BFF	sleep, rate limit, 95%+ usage
ok (green)	#6FCF97	success, General project
queued	#C2CDD8	queued
cancelled	#93A1B0	cancelled
offline	#5B6A7A	offline

- Each status color also has card, line and tint variants. **Model lanes:** agent (Claude Code) = coral text #FFA48C, tinted fill, squared radius 5; api = green text #8FE0B0, outlined, pill-shaped. **Project palette:** p1 #7FB2FF, p2 #FF7B5C, p3 #F58AC0, p4 #E6C07B, p5 #7FD1B9, plus muted and general #6FCF97. Diff colors for add, delete and hunk.

- **Radii:** 5, 8, 10, 12, 14, 16, 18, 20, 26 (sheets), 999 (pill). **Sizing:** tap target 44px; row height 56px (44 compact); section gap 20px; card padding 14px; page padding 14px (28px desktop); sidebar 250px; rail 60px; pane 460px; tab bar 96px. Shadows for drag, toast, menu, modal, pane and the coral FAB. Bar blur 14px.
- **Motion:** easing cubic-bezier(.2,.7,.3,1); durations .15s, .2s, .26s, .4s; 45 ms stagger; keyframes rise, fadein, spin, pulse, caret, shimmer, dot-bounce, load-bar, splash-out and the Octopus animations (wiggle, blink, bob, zz, dance, shadow).
- **Global:** dark color-scheme; thin scrollbars; 2px coral focus ring; placeholder color text-faint.

Empty, error and loading states (patterns to reuse)

- **Loading:** skeleton pages (shimmer), skeleton document list, "Loading settings…", "Loading the calendar…" with a spinner, "Putting your day together…", button spinners ("Approving…", "Retrying…", "Routing…", "Scheduling…", "Adding…", "Saving…", "Refreshing…", "Checking…"), a chat typing indicator.
- **Empty:** a reusable empty-state component (title, body, CTA): no projects, no activity, no schedules, no recipes, no documents; plus "Nothing queued", "Nothing finished yet", "Nothing yet. Write the first one in the box.", "No documents match.", "That project doesn't exist any more" with an "All projects" button; the notification panel's own empty text.
- **Errors:** failed task cards; Ask box "Could not send: …"; palette ETA line errors; the locked gate; the offline popup and banners; the paused-project banner and rate-limit card; "Octopus is restarting"; the fail() toast helper; invite error under a meeting; approval-link result pages.

7. Screen inventory checklist (everything to design)

1. Lock gate; Splash; Onboarding/first-run tour
2. App shell: desktop sidebar (expanded, collapsed rail, drawer), topbar (search, account switcher, usage pill, restart, bell), right-edge rail (Queue, Messages, Calendar), phone topbar, bottom tab bar, left and right drawers, status banners
3. Home (hero states, Octopus panel, rollups, Needs you, rate-limit, queue tabs, CRM card, Coming up, Follow-ups, Scheduled, Quick buttons, Usage, Models, Ask box and overlay)
4. Projects (Cards, Table, Board, filters, sort, bulk bar, empty states)
5. Project page with all 10 tabs (Overview/CRM record, Work, Backlog, Phases, Timeline, Notes, Documents, Activity, Recipes, Settings), CRM-only variant, paused variant, General variant, pinned-tab behavior
6. Clients list, Client page, Contact page, activity feed and log forms, follow-up forms, report download
7. CRM dashboard (counts, overdue, due week/month, at risk, follow-ups, pipeline, by client, stale, workload)
8. Recipes & schedules; Activity; Messages; Chat pane (docked, floating, minimised, inline, phone full screen); voice chat pane
9. Task detail page (plan steps, files changed, git, tests, approvals, diff, result); approval pane and approval-link result pages
10. Calendar; Briefing; Notes pad; Recycle bin; Hidden
11. Personal: Email, Calendar, Reminders, Connections (accounts, permissions, activity log)
12. Routing; AI Providers (models, API keys); Token usage; Voice (incl. Alexa section)

13. Settings: General (incl. email notifications), Octopus, Sites, Agents & tools, Calendar & voice, Shortcuts, Backup (export, check-then-confirm restore)
14. Meeting form with invite states; timeline item form; phase forms; AI generate-tasks sheet; phase report sheet
15. All sheets listed in 5.5; Spotlight; offline popup; confirm popovers; menus
16. Notification center (bell, toasts, panel) and system notification permission flow
17. Modules registry screen (new; not built)
18. Emails (HTML templates for the 9 events) and the approval-link result cards

8. Open items, gaps and design brief

- **Design brief:** build the complete Den UI from sections 2–7. Keep the phone-first approach and the 900px breakpoint, the dark theme with coral brand, mono-labelled dense cards, color reserved for status, and the Octopus mascot (six moods) in empty and error states. Keep the playful vs plain copy tones.
- **Pin-any-tab requirement (standing project note):** any tab on the project page can be pinned. Current code only pins chat tabs. Design a pin affordance on every project tab (the ten tabs plus Settings), a clear pinned state, and where pinned tabs sit (suggested: first in the strip, kept across projects, persisted per device or per user setting).
- **Not built / thin:** Outlook / Microsoft 365 (OAuth config exists; Graph adapters not built); local Whisper needs manual install; pictures on scheduled prompts; iOS Shortcuts flows (endpoints exist); Modules dashboard screen and command input; no client portal and no multi-user roles (by design today).
- **The README is out of date** versus the code: it lists four project tabs (Work, Activity, Recipes, Settings) while the code has ten plus Settings; it lists fewer nav items and shortcuts; and it describes toasts at bottom-centre while the code puts them top-right.
- **Survey caveats:** the screens Overview, Phases, Calendar, Personal, Routing, Providers, Usage, CRM table, Clients and Contacts were surveyed from code only in outline; their detailed layouts should be designed from the processes in section 4. The assistant package was described from the README. The density toggle's location was not found.
- **Security rule for the UI:** never display Claude tokens, OAuth tokens or full API keys; accounts.json holds live tokens in plaintext on disk.

9. Reference

9.1 API endpoints (all require X-API-Key except /health, the approval quick links, OAuth callback and /alexa)

Area	Endpoints
Health / service	GET /health; GET /service; POST /service/restart; GET /queue/status; GET|PATCH /settings; POST|DELETE /sleep; POST /notify/test; GET /dashboard, /dashboard/legacy
Projects	GET|POST /projects; PATCH|DELETE /projects/{id}; GET|PATCH /projects/{id}/settings; POST|DELETE /projects/{id}/image; POST /projects/{id}/suggest; POST /projects/{id}/report; GET|POST /projects/{id}/documents

Tasks	GET /tasks; POST /projects/{id}/tasks; GET /tasks/{id}/detail, /diff, /thread; POST /tasks/{id}/cancel, stop, pause, play, requeue, top, reviewed, read, summary; POST /queue/reorder; POST /messages/{id}/read; GET /messages
Backlog	GET|POST /projects/{id}/todos; PATCH|DELETE /todos/{id}; POST /projects/{id}/todos/reorder, /todos/run; PUT /todos/{id}/phase
Notes / documents	GET|POST /projects/{id}/notes; PATCH|DELETE /notes/{id}; GET|PATCH|DELETE /documents/{id}; GET|POST /pad; PATCH|DELETE /pad/{id}; POST /pad/{id}/save-file
Phases	GET /phases; GET|POST /projects/{id}/phases; POST /projects/{id}/phases/reorder; PATCH|DELETE /phases/{id}; POST /phases/{id}/generate-tasks, start, pause, complete, reopen; GET /phases/generation-models
CRM	GET /crm/dashboard; GET /crm/reports/{client|project}/{id}?format=json|csv|md; GET /crm/feed; GET|POST /clients; GET|PATCH|DELETE /clients/{id}; POST|DELETE /clients/{id}/logo; GET|POST /contacts; PATCH|DELETE /contacts/{id}; POST /projects/{id}/contacts; DELETE /projects/{id}/contacts/{cid}
Activities / follow-ups	POST /activities; PATCH|DELETE /activities/{id}; GET|POST /followups; PATCH|DELETE /followups/{id}
Timeline / briefing	GET /timeline; GET|POST /projects/{id}/timeline; PATCH|DELETE /timeline/{id}; GET /briefing
Approvals	GET /approvals; POST /approvals/{id}/decide; GET /approvals/{id}/context; GET /approvals/{id}/quick/{approve|deny}?t=
Octopus	POST /octopus; POST /octopus/plan; POST /octopus/plans/{id}/run, cancel; GET /routing, /routing/{id}; GET /octopus/overview, agents, tools, providers, models, usage, result/{task_id}, images/{name}; PATCH|PUT|DELETE|POST /octopus/models; POST /octopus/polish; POST /octopus/api-keys; DELETE /octopus/api-keys/{id}; POST /octopus/api-keys/{id}/default
Accounts / usage	GET|POST /accounts; DELETE /accounts/{id}; PUT /accounts/{id}/alias, color, details; POST /accounts/mac/show; POST /accounts/{id}/activate, refresh; GET /usage/summary, /usage/tokens
Recipes / schedules	GET|POST /recipes; DELETE /recipes/{id}; POST /recipes/{id}/run; GET|POST /schedules; PATCH|DELETE /schedules/{id}
Uploads / voice	POST /uploads; GET /uploads/{id}; GET /voice; POST /voice/transcribe, speak, command; POST /alexa
Notifications	GET /push/key; POST|DELETE /push/subscribe; POST /presence; GET|PATCH /email; POST /email/test
Personal assistant	GET /assistant/status; POST /assistant/connections/{provider}/connect; GET /assistant/oauth/{provider}/callback; DELETE /assistant/connections/{provider}; GET|PATCH /assistant/policies; POST /assistant/route; GET /assistant/activity; GET /assistant/email/overview, calendar/overview; GET|DELETE /assistant/reminders
Backup	GET /export/{kind}?format=; GET /backup; POST /restore/check; POST /restore
Modules	GET /modules; POST /modules/rescan; GET /modules/commands; POST /modules/commands/{name}; GET /modules/{name}; POST /modules/{name}/enable|disable|reload; PATCH /modules/{name}/config

9.2 Settings keys worth surfacing

- `notify_on_resume`, `hold_low_when_busy`, `monthly_budget_usd`, `sleep_mode`, `email_to`, `email_events_off`, `briefing_time` (08:30), `briefing_push`, `voice_rate`, `voice_language_voices`, `voice_manual`; project settings `auto_in_progress`, `autopilot`, `autopilot_ceiling`, `budget_usd`, `claude_account`, `notify_approvals/failures/completions`, `quiet_hours`.

- Browser-local: `cq_api_key`, `octopus-pinned-tabs`, `octopus-shortcuts`, `octopus-last-seen`, `octopus-compact`, saved panel widths, browser voice choices.

9.3 Key files

- Backend: api.py (about 160 routes), db.py (tables and CRM columns), worker.py (queue, notifications, autopilot, reminders, briefing), router.py, crm.py, crm_reports.py, clients.py, activities.py, timeline.py, invites.py, phases.py, mailer.py, email_templates.py, approval_links.py, approval_server.py, risk.py, webpush.py, presence.py, accounts.py, api_keys.py, documents.py, pad.py, project_images.py, titles.py, backup.py, restart_control.py, octopus/ (plan, control, execution, service, voice, models), assistant/ (intent, planner, actions, scope, capabilities, tools, providers, oauth), alexa/, den_modules/, modules/hello, models.json, agents.json, tools.json, mcp-config.json.
- Front end: dashboard.html, static/js/app.js, static/js/views/ (home, shell, project, projects, chat, sheets, queue-panel, status-banner, offline, splash, settings, crm-*, clients, calendar, personal, routing, providers, usage, voice-pane, and others), static/js/notifications.js, gestures.js, shortcuts.js, voice.js, static/css/tokens.css, mobile.css, static/sw.js, design_handoff_tushars_workspace/ (the original design spec).
- Docs: README.md, docs/MODULES.md, setup/ (launchd plist, Alexa interaction model and manifest).

End of document.